MongoDB Functions & Operators

A complete reference: what each type does, why it exists, and where it's used

MongoDB does not have "functions" in a single flat list — its functionality is organized into several distinct categories, each serving a different purpose in the database workflow: connecting to data, querying it, shaping it, transforming it in bulk, and administering the server itself. This document groups every major category together, explains **why** it exists (the problem it solves) and **where** it is used (the typical stage of a MongoDB application it belongs to), with the most commonly used operators/methods listed in tables for quick reference.

Contents

1. CRUD Methods

Why it exists: These are the foundational methods for Create, Read, Update, and Delete operations — the basic actions any application needs to persist and retrieve data.

Where it's used: Application code (Node.js/Python/Java drivers), the `mongosh` shell, and backend APIs that talk directly to a MongoDB collection.

Function	Purpose	Typical Use
insertOne()	Adds a single document to a collection	Saving one new user, order, or record
insertMany()	Adds multiple documents in one call	Bulk-importing seed data or batch records
find()	Retrieves documents matching a filter	Listing/searching records, building reports
findOne()	Retrieves the first matching document	Fetching a single record, e.g. login lookup
updateOne()	Modifies the first matching document	Editing one user's profile field
updateMany()	Modifies all matching documents	Bulk status changes across many records
replaceOne()	Replaces an entire document	Overwriting a document with a new version
deleteOne()	Removes the first matching document	Deleting a single record
deleteMany()	Removes all matching documents	Bulk cleanup, e.g. removing expired sessions
findOneAndUpdate()	Finds, updates, and returns the document	Atomic read-modify-write, e.g. queue pop
bulkWrite()	Executes mixed write operations in one batch	High-throughput batch processing pipelines

2. Query Operators

Why it exists: Plain equality matches aren't enough for real filtering — query operators let you express comparisons, ranges, logical combinations, and existence checks inside the filter document.

Where it's used: Inside the filter argument of `find()`, `updateOne()`, `deleteMany()`, and the `$match` aggregation stage.

Operator	Purpose	Example Use
$eq / $ne	Equal to / not equal to a value	status = 'active'
$gt / $gte	Greater than / greater or equal	price > 100
$lt / $lte	Less than / less or equal	age <= 18
$in / $nin	Value is (not) in a given array	category in ['books','toys']
$and / $or / $nor	Combine multiple conditions logically	price < 50 AND inStock = true
$not	Negates a condition	Excluding matches, e.g. NOT $regex
$exists	Checks whether a field is present	Finding docs missing an 'email' field
$type	Matches by BSON data type	Finding docs where 'age' is stored as a string
$regex	Pattern-matches string fields	Case-insensitive text search on names

Operator	Purpose	Example Use
$elemMatch	Matches array elements against criteria	Order where any item has qty > 5
$size	Matches arrays of a specific length	Carts with exactly 3 items
$all	Array contains all specified values	Docs tagged with both 'sale' and 'new'

3. Update Operators

Why it exists: Updates need to modify fields in place — increment counters, push to arrays, rename fields — without requiring the client to fetch, edit, and rewrite the whole document.

Where it's used: Inside the update document of `updateOne()`, `updateMany()`, and `findOneAndUpdate()`.

Operator	Purpose	Example Use
$set	Sets/overwrites a field's value	Updating a user's email address
$unset	Removes a field entirely	Deleting a deprecated field from a document
$inc	Increments/decrements a numeric field	Incrementing a page view or stock counter
$mul	Multiplies a numeric field	Applying a discount multiplier to price
$rename	Renames a field	Schema migrations without full rewrites
$min / $max	Updates only if new value is lower/higher	Tracking a record low/high score
$push	Adds an item to an array	Adding a comment to a post's comment array
$pull	Removes matching items from an array	Removing a tag from a tags array
$addToSet	Adds item only if not already present	Preventing duplicate entries in an array
$pop	Removes first/last element of an array	Implementing a queue or stack structure
$currentDate	Sets a field to the current date	Auto-updating an 'updatedAt' timestamp

4. Aggregation Pipeline Stages

Why it exists: Simple queries can't reshape, join, or summarize data. The aggregation pipeline chains stages together so documents flow through a series of transformations, similar to SQL's GROUP BY, JOIN, and multi-step reporting queries.

Where it's used: Inside `collection.aggregate([...])` — used heavily for analytics dashboards, reports, and denormalized data joins.

Stage	Purpose	Example Use
$match	Filters documents (like find())	Narrowing to this month's orders
$group	Groups documents & computes aggregates	Total sales per region
$project	Reshapes documents / selects fields	Returning only name and computed total

Stage	Purpose	Example Use
$sort	Orders documents	Ranking top-selling products
$limit / $skip	Paginate results	Showing page 2 of search results
$lookup	Performs a left-outer join to another collection	Joining orders with customer details
$unwind	Flattens an array into separate documents	Reporting on each item inside an order
$addFields	Adds computed fields without dropping others	Adding a 'fullName' derived field
$count	Counts documents passing the pipeline so far	Getting total matching record count
$facet	Runs multiple sub-pipelines in parallel	Building a search page with counts + results
$bucket	Groups documents into value ranges	Histogram of orders by price range
$merge / $out	Writes pipeline results to a collection	Materializing a reporting table

5. Aggregation Expression Operators

Why it exists: Within pipeline stages you often need to compute values (sums, string concatenation, conditionals) rather than just move documents around — these operators do that math and logic.

Where it's used: Inside `$group`, `$project`, and `$addFields` stage expressions.

Operator	Purpose	Example Use
$sum	Adds numeric values (also counts docs)	Total revenue per category
$avg	Calculates the average	Average order value
$min / $max	Finds smallest/largest value in a group	Cheapest/most expensive product per brand
$first / $last	First/last value seen in a group	Most recent status per user, after $sort
$concat	Joins strings together	Combining first + last name
$cond	If/then/else conditional expression	Labeling orders as 'high' or 'low' value
$ifNull	Returns a fallback if field is null/missing	Defaulting missing 'nickname' to 'name'
$dateToString	Formats a date field as a string	Grouping sales by 'YYYY-MM'
$toUpper / $toLower	Changes string casing	Case-insensitive grouping/reporting
$arrayElemAt	Gets an element at a given array index	Extracting the first image URL
$map	Transforms each element of an array	Applying a formula to every line item
$reduce	Folds an array down to a single value	Custom accumulation logic across an array

6. Array Query & Projection Operators

Why it exists: Arrays are a first-class MongoDB data type, so dedicated operators exist to query and shape them precisely instead of treating them as opaque blobs.

Where it's used: Filter documents (query stage) and the projection document of `find()` / `$project`.

Operator	Purpose	Example Use
$elemMatch (query)	Match docs where one array element meets several conditions	Order with an item priced > 50 AND qty > 1
$elemMatch (projection)	Return only the first matching array element	Fetching just the matching review from a list
$slice	Returns a subset (slice) of an array	Paginating comments within a document
$ (positional)	References the first array element that matched the query	Updating the matched item inside an array
$[] (all positional)	References all elements of an array in an update	Applying a discount to every item in a cart
$[] (filtered positional)	References array elements matching an arrayFilter	Updating only 'pending' items within an order

7. Cursor Methods

Why it exists: find() returns a cursor, not the final result set — cursor methods let you refine, order, and limit results lazily before the data is actually pulled from the server.

Where it's used: Chained directly onto the result of `find()`, e.g. `db.orders.find().sort().limit()`.

Method	Purpose	Example Use
sort()	Orders the returned documents	Newest orders first
limit()	Caps the number of documents returned	Top 10 results
skip()	Skips a number of documents	Pagination, combined with limit()
project() / .find(filter, projection)	Selects which fields to return	Returning only name & email, not full document
count() / countDocuments()	Counts matching documents	Showing 'X results found'
forEach()	Iterates over each document in the cursor	Custom processing per document in mongosh
toArray()	Converts the cursor into an in-memory array	Loading all results into application memory
hint()	Forces use of a specific index	Query performance tuning/debugging

8. Index & Performance Methods

Why it exists: Without indexes, MongoDB scans every document to satisfy a query. These methods create and manage indexes so lookups, sorts, and filters run in milliseconds instead of seconds.

Where it's used: Run occasionally during schema/deployment setup, and inside performance-tuning or migration scripts.

Method	Purpose	Example Use
createIndex()	Builds a new index on one or more fields	Speeding up lookups by 'email'
dropIndex()	Removes an existing index	Cleaning up an unused/expensive index
getIndexes()	Lists all indexes on a collection	Auditing what indexes currently exist
explain()	Shows the query execution plan	Diagnosing why a query is slow
createIndex({..., unique:true})	Enforces uniqueness on a field	Preventing duplicate usernames/emails
createIndex({..., expireAfterSeconds})	TTL index — auto-deletes old documents	Expiring session tokens or logs automatically

9. Administrative / Database Methods

Why it exists: Beyond data operations, MongoDB needs methods for managing the database itself — creating collections, checking server status, managing users, and handling transactions.

Where it's used: Run from `mongosh`, admin scripts, or backend setup/migration code — not typically inside everyday application request handlers.

Method	Purpose	Example Use
db.createCollection()	Explicitly creates a collection (with options/validation)	Setting up a schema-validated collection
db.collection.drop()	Deletes an entire collection	Cleaning up test/staging data
db.stats() / collection.stats()	Reports storage & performance statistics	Monitoring database size and growth
startSession()	Starts a client session for transactions	Multi-document ACID transactions
session.startTransaction()/commitTransaction()	Groups multiple writes atomically	Transferring funds between two accounts
db.createUser()	Creates a database user with roles	Setting up application/service credentials
db.currentOp()	Shows currently running operations	Diagnosing a stuck/slow long-running query
watch()	Opens a change stream on a collection	Real-time sync, notifications, event pipelines

10. Evaluation, Bitwise & Text-Search Operators

Why it exists: Some filtering needs go beyond comparisons — validating against a JSON schema, doing bit-level checks, or running full-text search — so MongoDB provides specialized operators for these cases.

Where it's used: Inside query filters and the `$text`/`$search` stages, typically for validation, search features, and low-level flag fields.

Operator	Purpose	Example Use
$expr	Uses aggregation expressions inside a query	Comparing two fields of the same document
$jsonSchema	Validates documents against a JSON schema	Enforcing required fields/types on insert
$mod	Matches values by a modulo operation	Finding every 5th record for sampling
$bitsAllSet / $bitsAnySet	Checks bits set in a numeric field	Permission/flag fields stored as bitmasks
$text	Performs full-text search (needs a text index)	Searching articles by keyword
$search (Atlas Search)	Full-text/fuzzy search via Atlas Search index	Autocomplete and relevance-ranked search
$geoWithin / $near	Geospatial queries	Finding stores within a delivery radius

Note: This reference focuses on the most frequently used functions and operators across a typical MongoDB application lifecycle — from writing data, to querying and updating it, to building analytics with the aggregation framework, to administering the database. MongoDB's full documentation lists further specialized operators (e.g. more geospatial, more bitwise, encryption-related fields) for advanced use cases not covered here.